Lecture 24 · Part VI · Lecture notes

Generation, retrieval-augmented systems, and where this leaves you

Dataset: COCO and the Part V corpora

How to use these notes

The primary text for this lecture is Géron, Chapters 15–16. These notes are not a replacement for it and do not repeat it: they are the lecture's own argument written out at length — the derivation done slowly, the numbers with the runs that produced them, and the reasoning between one slide and the next that a deck can only gesture at. Read the chapter for the method; read these for why the lecture makes the choices it does. Every figure quoted here is printed by this lecture's notebook, and the course checks that mechanically rather than trusting it.

Contents

1 Two weak spots, both about entries **2**

2 Filling the holes **2**

2.1 The verdict on the joint space 3

3 Retrieval-augmented generation **3**

3.1 The ambiguous query 4

3.2 Decide how you will check it first 4

4 The course, as one argument **5**

4.1 The seventeen derivations, in one place 6

4.2 The failures, collected 6

4.3 The catalogue of silent failures, closed 6

4.4 What this course did not cover 7

5 What you can do now **7**

6 The notebook **8**

7 Design exercise: a developer-tools company's documentation assistant **8**

8 Exercises **11**

Summary **11**

1 Two weak spots, both about entries

Lecture 23 built a catalogue you can search with a sentence. Two encoders trained apart put text and images in spaces with nothing between them, and ranked no better than chance; one dual encoder, trained contrastively on pairs, on the unit sphere and with a temperature, put the right entry first for 75.0% of 200 queries. Two things remain, and both are about entries rather than about the model. Some entries have *no usable description*, so there is nothing for a text index to encode — and a text index cannot rank a document that does not exist, so those entries score exactly 0%, structurally rather than statistically. And some queries are *ambiguous*, where a ranking is a poor answer to a question the user has not finished asking.

Both are repaired by generation. A multimodal model can write the missing description; and a generator with the retriever attached can answer the ambiguous query in words, grounded in what the catalogue actually contains.

2 Filling the holes

60 of the 200 entries have no description, and score exactly 0% on the text route. We have a model that maps images to text in the retrieval direction; use one built for *generating* the text instead, and write the missing descriptions. The captioner is architecturally the encoder–decoder of Chapter 15 with an image encoder in place of the text one — a vision transformer, a transformer decoder cross-attending to it, trained on image–caption pairs by next-token prediction, 224 million parameters. Nothing new; a different pair of modalities in the same box.

On the 60 affected entries	R@1
Text index, description written by a person	66.67%
Text index, description deleted	0%
Text index, auto-captioned	60.00%

From 0% to 60.00% on the 60 affected entries, against 66.67% had a person written them: the caption recovers *part* of the loss, not all of it, and the part is what to report. (Two decimals, because on 60 entries each one is worth more than a point and a half.) The joint-space image route, which never needed a description, sits above all three bars in the deck's figure and is untouched by the repair.

2.1 The verdict on the joint space

Both repairs lean on Lecture 23's joint space, which nothing in this lecture touches. For the first time in the course, a first guess at its number was probably *too low* rather than too high — because the failure was not in the evaluation but in the geometry, and the repair was somebody else's pretraining run.

Failure condition — 75.0% over 200 is not 75.0% over fifty thousand

Recall rises as the haystack shrinks, and a catalogue is a haystack that grows. The same model over fifty thousand candidates would report a lower number without having changed, so the figure does not transfer beyond its candidate-set size.

3 Retrieval-augmented generation

The user asks in words; the retriever of Lecture 20 embeds the question and returns the k most similar entries; those go into the generator's context with the question; and the generator answers *from what it was given*, citing which entry it used.

Why this is not just a bigger model: *the knowledge lives in the index, not in the weights*. Add a product and it is answerable immediately; remove one and it stops being cited. No retraining — and the system can say where an answer came from, which is usually the requirement that decides whether it can be deployed at all.

	Good at	**Bad at**
Retriever	finding the right entries, fast, over millions	saying anything; it only ranks
Generator	fluent answers, synthesis across entries	knowing anything reliably; it will produce a plausible answer regardless

Each covers the other's weakness, which is why the combination is worth more than either — and why its failures are interesting: they belong to *neither half*.

Failure condition — visible from neither half

Retrieved but ignored — the right entry is in the context and the generator answers from its own weights instead.
Not retrieved, answered anyway — the retriever misses and the generator does not say so, producing a fluent answer with nothing behind it.
Retrieved and misread — the entry is present and correct, and the answer misstates it.
Evaluating the retriever alone by recall@k and the generator alone by fluency can both look excellent while the system does all three. *The join needs its own measurement.*

Two upstream decisions are usually made without measurement. *Chunking*: too small and a chunk lacks the context that makes it answerable; too large and the embedding averages several topics and matches none well; split badly and a sentence cut in half retrieves for neither of its halves. It is a hyperparameter — vary it, hold everything else fixed, and measure recall@k on

the same judgements. And *how many to retrieve*: small k risks the right entry not being there at all, large k buries it among distractors — which is where re-ranking earns its cost, retrieving generously with the cheap bi-encoder and re-ranking with the cross-encoder, high recall from the first stage and high precision from the second.

Four ways to measure the join: *groundedness*, is every claim supported by a retrieved entry; *attribution*, does the cited entry actually say it — a citation that does not support its claim is worse than none; *abstention*, when nothing relevant is retrieved does it decline, measured deliberately with queries whose answer is genuinely absent; and only then, end to end, is the answer right. Plus the closed-book control: run the same questions with retrieval off, and whatever the system still answers correctly it knew already. *The difference is what retrieval actually bought* — the same shape as every ablation in this course.

3.1 The ambiguous query

Customers type "something to sit on", "gear for bad weather", "something to put flowers in". A ranked list of pictures is a poor answer to any of these: the customer wants a shortlist with reasons, and a ranking has no place to put a reason. Six such queries are written by hand, and the counts are the notebook's run over them — there is no dataset of ambiguous queries, and inventing them and saying so is the honest option.

Two options do not work. Show the top ten pictures and let the customer decide — that is what we already have, and it is why they asked in words. Or ask a language model — it has never seen our catalogue, so anything it recommends is invented, and it will invent it fluently. Retrieval-augmented generation is the composition of the two: *retrieve first, then generate strictly from what was retrieved*.

3.2 Decide how you will check it first

"The answer is good" is not measurable in a lecture. So give every entry a stock number and require the assistant to cite them: a cited SKU either exists in the catalogue or it does not. Count the citations, count the ones that exist, report the fraction — no judgement call anywhere in the loop.

This measures grounding, not helpfulness. Say which one you measured; they are not the same thing and only one of them is cheap.

The pipeline is short: embed the query in the joint space, take the top five catalogue images, put their SKUs and descriptions into the prompt with the instruction to cite only SKUs from that list — and, if none fit, to say so — then extract every `CAT-` identifier from the answer and count how many are in the catalogue.

Over 6 ambiguous queries	**Closed book**	**Retrieval-augmented**
Citations emitted	21	12
Citations that exist	0	12
Grounding rate	0%	100%

Closed book, the model produces a fluent recommendation and cites stock numbers *in exactly*

the right format. Across six queries it emitted 21 citations, of which none corresponds to an entry that exists.

The failure is not that it refuses

It is that it does *not* refuse, and nothing in the output distinguishes an invented SKU from a real one.

Failure condition — what this does not fix

Grounding is not correctness — a cited entry can exist and still be a bad recommendation. We measured the cheap half.

The retriever is now the ceiling: if the right entry is not in the top five, no amount of generation recovers it, which is why the R@5 measured last lecture is the number that matters here.

And fluency is unchanged. A wrong answer built from real SKUs reads better than a wrong answer built from invented ones.

Evaluating generated answers — faithfulness, attribution, judging with another model — is a live research area and entirely outside Chapters 1–16: not examinable, and worth knowing exists.

Route	Before	After
Text query to image, joint space	75.0%	75.0% (untouched)
Text index, on the entries with no description	0%	60.00%
Ambiguous queries, grounding rate	0%	100%

Two repairs, two measured deltas, and *one number deliberately unchanged as a control*. That last row is what makes the other two believable.

4 The course, as one argument

The argument

Every method in this course arrived because something measurable could not be done without it.

And every lecture had the same shape: a quantity that has to be derived before the method makes sense; the method; a worked example with real numbers; and the conditions under which it fails. Partway through, the lectures were cut to what the room actually covered — fewer slides each, and the same four parts in every one. Each application broke first on a measurement, and the method arrived afterwards: that sequence, not the chapter order, is the syllabus.

4.1 The seventeen derivations, in one place

Least squares and the normal equation	2, 3	Weight sharing and equivariance	12
Imbalance; precision is not monotone	3	IoU's vanishing gradient; mAP	14
Gradient descent	4	Stationarity, differencing, autocorrelation	15
Variance of a correlated average	5	Softmax, cross-entropy and logits	17
PCA via the SVD	6	Scaled dot-product attention	18
What a layer computes	7	Evaluating a ranking: MRR, AP, NDCG	19
Reverse-mode automatic differentiation	8	Matrix factorisation and the SVD	21
The Shapley value and its efficiency property	10	The contrastive objective and its temperature	23
A linear autoencoder recovers the principal subspace	11		

Forty per cent of the written paper, and the part of this course with the longest shelf life.

4.2 The failures, collected

Failure	**First met**	**Measured cost**
Fitting before splitting	Lecture 3	nothing measurable — and unknowable in advance
Evaluating on training data	Lecture 3	an RMSE of zero
Accuracy under imbalance	Lecture 3	a useless model above 90%
Choosing on the test set	Lectures 3, 5	an optimistic report, every time
Missing `zero_grad()`	Lecture 8	66 points
Ten convolutions, no batch norm	Lecture 12	a network that does not train at all
Random split on a series	Lecture 15	optimistic, and undeployable
Augmenting the validation set	Lecture 13	2.16 points of noise on the criterion

Not one of them raises an exception. That is the whole reason the course is organised around measurement.

4.3 The catalogue of silent failures, closed

It runs, and produces a plausible number	**Diagnosed in**
Scaler fitted before the split	L3
Evaluating on training data	L3
Accuracy under class imbalance	L3
Hyperparameters chosen on the test set	L3, and again on a prompt template in L23
Missing `model.eval()` / `optimizer.zero_grad()`	L8
Metric averaged per batch, not over the set	L8
Random split on an autocorrelated series	L15
Embeddings compared without normalising	L23

Every row of that table runs to completion. Not one of them raises.

4.4 What this course did not cover

Missing, and each is a course: *reinforcement learning*, an agent acting and learning from the consequences — a different problem shape entirely; *generative image models*, diffusion and what it optimises; *causality* — every method here predicts, and none tells you what happens if you *intervene*; *fairness and model governance*, who a model is worse for and who is accountable when it is; and *serving and drift*, the model in production and how it decays quietly once the world moves. They are named so you know they exist and roughly where they sit. None is examinable.

The two you are most likely to need first

Drift. Every number in this course was measured on data drawn like the training data. In production that stops being true, gradually and without warning, and the score you reported at launch is the last honest one anybody has. Monitor the inputs, not just the outputs — the distribution moves before the accuracy does.
Who it is worse for. Lecture 22 asked whom a recommender serves: popularity alone scores respectably, and the metric gives a film everyone has heard of the same credit as a discovery. Asking that costs a line of code, and it is the difference between a system you can defend and one you cannot. Always ask for whom the average is wrong.

Beyond the book	
Causal inference	every model here answers "what is associated with what"; deciding an intervention needs different assumptions and different data
Uncertainty quantification	Lecture 4 checked whether one classifier's probabilities were calibrated; predictive uncertainty in general is its own subject
Fairness and harm	we asked once, in Lecture 22, whom a system serves; who it fails, and what that costs them, is not a metric you can pick off a list
Privacy	differential privacy, federated learning, and what a model memorises about its training data
Monitoring and drift	a model is fitted once and lives for years; detecting that the world moved is a permanent job
Experimental design	an offline metric improving is not a business improving; that is an A/B test, and a statistics problem

5 What you can do now

Take a problem, frame it, and say what a correct answer would be before fitting anything. Choose a metric that matches the brief, and measure the trivial baseline first. Build the simplest thing that runs, inside a pipeline, evaluated on a protocol fixed in advance. Read the failure — slice the error, look at the worst cases, name the conditions under which the method breaks. Improve one thing at a time and report the ablation rather than the best row. And write down what the number was measured on, every time.

Nothing in that is specific to a library, and none of it will be out of date in ten years.

A last word on the assistant

Much of the code you write from here on will be drafted with an assistant's help. That changes who types it. It does not change what this course was about.

What an assistant cannot supply is whether the number at the end means anything. "What was this measured on?", "what is the baseline?", "was anything fitted before the split?" — those questions come from knowing the material, and every lecture of this course was built around them.

The code was never the scarce part. Knowing whether to believe the output is, and that is what these twenty-four lectures were for.

How to keep learning: reproduce something — take a paper's headline number and try to get it, and you will learn more from the gap than from the paper. Keep the habit of the baseline — the single cheapest defence against a wrong conclusion, and almost nobody does it. Read the evaluation section first: if it does not say what was held out and how, the rest is decoration. And write the check before the model; every notebook in this course does, and it is why they still run.

The libraries in these notebooks will be superseded. The derivations will not, and neither will the habit of asking what a number was measured on.

6 The notebook

What to change

Set `do_sample=True` and run the grounding measurement three times. The percentage moves every run: a sampled generation makes the grounding rate a random variable, and nothing in the notebook has characterised its spread.

7 Design exercise: a developer-tools company's documentation assistant

The lecture closes with a design exercise in the manner of an ML system-design interview, on a product with the shape of data this model suits.

The task. A developer-tools company sells a payments API. On its documentation site a developer asks "how do I rotate a key without downtime?" and the assistant must answer in words, from the company's own documentation, citing the pages it used — or say that the documentation does not answer. The pages change at every release, and some describe features only certain customers may see. The business measures support tickets deflected, the time to a first successful call, and the cost of a developer shipping what a wrong answer said. Given: the documentation — guides, reference pages, changelogs, each with an identifier, an owner and a visibility — and the support channel's questions, a sample judged by the writers against the page that answers. Tens of thousands of pages, thousands of questions a day. The output goes to the developer asking, the support team, and the writers who own the pages.

1. **Frame it.** "The answer is good" is not measurable. What is: every citation names a page that

exists; the cited passage supports the sentence it is attached to; and when nothing relevant is retrieved, the assistant declines. Two baselines, both bad on purpose: show the top ten passages and let the developer read, which is what the docs search already does; and ask a language model with no documentation, which is fluent and invented. End-to-end correctness is judged last, by the writers, on a sample. *Pitfall:* measuring grounding and reporting it as correctness. A cited page can exist, support the sentence, and still be the wrong procedure for the developer's API version, and the launch post quotes the number. Grounding is the cheap half and the half a machine can check; say which half a number measures, every time.

2. **Chunking.** Before anything is retrieved the documentation is cut into units, each carrying its page's identifier, title, owner and visibility — a choice upstream of both halves, usually made without measurement. Too small and a chunk lacks the context that makes it answerable; too large and its embedding averages several topics and matches none; split badly and a code sample is cut from the sentence that explains it. The judged questions are a sample from the support channel, each marked by the page's writer with the page that answers it, or with the fact that none does. *Pitfall:* picking a chunk size by feel and never revisiting it. Vary it with everything else held fixed and measure recall at k on the judged questions — the retrieval protocol, unchanged — then cut at the page's own headings, so that a procedure's steps and its code sample stay together and a chunk can be shown as a readable passage.

3. **The split, and the prompt.** Validation questions for every choice — chunk size, how many to retrieve, the re-ranking depth, the wording of the prompt — and test questions read once, before the release. Include questions whose answer is genuinely absent from the documentation, in both halves, so that abstention is measured deliberately rather than hoped for; and fix the decoding, because a sampled generation turns every rate into a random variable, so decode deterministically or report the spread over repeated runs. *Pitfall:* the template chosen on the test set. Trying several prompt wordings and reporting the one with the best grounding rate on the test questions is a hyperparameter chosen on the test set, with a sentence in place of a number — the same error met with a numeric hyperparameter and again with a zero-shot prompt. Choose the template on the validation questions; report it on the test, once.

4. **The retriever, and the ceiling.** A bi-encoder over the chunks, fused with a lexical index for the endpoint names and error codes a vector space does not guarantee to match, and a cross-encoder re-ranking the top: retrieve a generous k with the cheap first stage and pass only the top few on, high recall from the first stage and high precision from the second. Measure the retriever on its own, by recall at the k the generator will see, on the judged questions. *Pitfall:* tuning the generator while the right chunk is not in the context. If the page that answers is not among the k retrieved, no amount of generation recovers it: the retriever's recall at k is the ceiling of the whole system, and a quarter spent on prompts under a low ceiling is lost. Read that number first, and spend on the retriever — chunking, hybrid, re-ranking — before touching the prompt.

5. **The join.** The retrieved chunks go into the generator's context with the question and an instruction: answer from these pages only, cite their identifiers, and say so if none of them answers. The knowledge lives in the index, not in the weights — a page added at a release is answerable at once, a deprecated one stops being cited, nothing is retrained — and each half covers the other's weakness: the retriever only ranks, the generator will answer regardless. *Pitfall:* three failures visible from neither half. Retrieved but ignored: the right page is in

the context and the answer comes from the weights, an older API. Not retrieved, answered anyway: the retriever misses and the generator does not say so. Retrieved and misread: the page is present and correct and the answer misstates it. Recall at k and fluency can both look excellent while the system does all three; the join needs its own measurement.

6. **The closed-book control.** Extract every identifier the answer cites and count those that exist: the grounding rate, with no judgement call in the loop; then, on a sample, whether each cited passage supports its sentence — attribution, checked by a writer. Run the same questions with retrieval off: whatever the assistant still gets right it knew already, from the old docs on the public web, and the difference is what retrieval bought. On the absent-answer questions, count the abstentions. *Pitfall:* a citation that looks right. Closed book, the model produces a fluent answer and cites pages in exactly the right format, none of which exists; nothing in the output tells an invented citation from a real one, and a developer without the docs open cannot tell either. The check has to be mechanical — identifier against the index, then passage against the claim — and a citation that does not support its sentence is worse than none.

7. **Access control.** Every chunk carries its page's visibility: public, private beta, or internal. The retriever filters by the asker's entitlements before ranking, so that a chunk the developer may not read never reaches the generator, because the generator cannot keep a secret it has been shown: whatever is in the context can appear, quoted or paraphrased, in the answer. Test it deliberately, with questions whose answer lies only in a page the asker cannot see, where the required behaviour is to decline. *Pitfall:* leakage into the answer. One index over everything, filtered after generation or not at all: an unannounced feature from a private-beta page is retrieved because it matches the question well, and the assistant describes it, correctly grounded, to a customer who should not know it exists. Filter at retrieval, log which pages each answer was built from, and keep the cannot-see questions in the test set, so that a leak is a failed test rather than an incident.

8. **In production.** Re-index on every release, and re-run the judged questions after each rebuild — recall at k, grounding, abstention — a drop being the regression and where a renamed page shows up. Roll out to a share of developers first, and monitor the live answers: the grounding rate by day, citations to pages that no longer exist, the abstentions, and the tickets that follow an answer, with an engineer on call for the index. Route the questions with no supported answer to the writers: they are the list of what the documentation does not say. *Pitfall:* treating the launch number as permanent. The score reported at launch was measured on that release's documentation; pages are rewritten, moved and withdrawn, the questions drift with the product, and the assistant decays quietly because nothing raises an exception when a cited page stops existing. Monitor the inputs and the citations, and refresh the judged set as the documents are.

9. **Ship it.** Documentation → chunks at the headings, each with its identifier and visibility → judged questions split, absent answers included → hybrid retriever, re-ranked, recall at k first → generator answering from the context only, with citations → grounding checked mechanically, closed-book control → entitlements filtered at retrieval → re-index at every release, roll out, monitor.

8 Exercises

The five set at the end of this lecture's deck, in the style of the written paper. The solutions are at the end of this lecture's deck and in the exercise book.

1. Sixty catalogue entries have no description. State why their text-route R@1 of 0% is structural rather than statistical, and what fraction of the loss an auto-caption recovered. [5]
2. Name the three failures that belong to the join of a retriever and a generator, and state why evaluating each half alone does not see them. [5]
3. A retrieval-augmented system is measured by whether cited stock numbers exist. State what that measures, and what it does not. [4]
4. In a retrieval-augmented pipeline the retriever's recall bounds the whole system. Explain why, and what that implies about where to invest. [4]
5. Name three rows of the course's catalogue of silent failures, and state the property they share and what it implies about how such failures are found. [5]

Summary

Generation repaired the two holes Lecture 23 left. Captioning took entries that were structurally invisible to a text index — not badly ranked, absent — and made them findable: R@1 on those 60 entries went from 0% to 60.00%, against 66.67% had a person written them. The joint-space image route, which never needed a description, was the control nothing touched.

Retrieval-augmented generation took six ambiguous queries from a 0% grounding rate to 100%. Closed book, the model emitted 21 citations in exactly the right format, of which none existed — and the failure is not that it refuses but that it does not, with nothing in the output distinguishing an invented identifier from a real one. Grounding is not correctness; we measured the cheap half, and said so.

The course was one argument: every method arrived because something measurable could not be done without it. Seventeen derivations, and a catalogue of failures none of which raises an exception — a scaler fitted early that cost nothing measurable, a missing `zero_grad()` that cost 66 points, a random split on a series that was optimistic and undeployable.

The closing design exercise built a developer-tools company's documentation assistant in nine steps — the check decided first and grounding kept apart from correctness, chunking measured as a hyperparameter, the prompt chosen on validation, the retriever's recall read as the ceiling, the three failures of the join, a mechanical citation check with a closed-book control, entitlements filtered at retrieval, and the judged set re-run at every release — each step named for the pitfall it avoids.

Failure condition — a citation that looks right

A generated answer can cite a SKU that does not exist, or one that exists and does not support the sentence. Both read as citations. The check has to be mechanical — match the SKU against the catalogue, then the passage against the claim — because a reader without the catalogue cannot tell.

If you remember one thing

A number is only as good as the question "measured on what?" can be answered. Every failure in this course — the leak, the training score, accuracy under imbalance, the random split on a series, the augmented validation set, the ungrounded generated answer — is a case where that question had an answer nobody had asked for. Asking it costs seconds.